

GymPro — MySQL SELECT Commands

Run in MySQL Workbench to verify every Postman action • Base URL: <http://localhost:8080>

Auth service

gympro_auth

All registered users

```
USE gympro_auth;
SELECT id, name, email, role FROM users ORDER BY id;
```

Check a specific user by email

```
SELECT id, name, email, role FROM users
WHERE email = 'admin@gympro.com';
```

Count users per role

```
SELECT role, COUNT(*) AS total FROM users GROUP BY role;
```

Member service

gympro_member

All member profiles

```
USE gympro_member;
SELECT id, name, email, phone, gender, status FROM members ORDER BY id;
```

Active members only

```
SELECT id, name, email, status FROM members
WHERE status = 'ACTIVE';
```

Verify member was deactivated (DELETE endpoint)

```
SELECT id, name, email, status FROM members
WHERE status = 'INACTIVE';
```

Trainer service

gympro_trainer

All trainers

```
USE gympro_trainer;
SELECT id, name, email, specialization, experience_years, status
FROM trainers ORDER BY id;
```

All trainer schedule slots

```
SELECT id, trainer_id, day_of_week, start_time, end_time, available
FROM trainer_schedules ORDER BY trainer_id, day_of_week;
```

Available slots only (not yet booked)

```
SELECT id, trainer_id, day_of_week, start_time, end_time
FROM trainer_schedules WHERE available = 1;
```

Slots for a specific trainer (e.g. trainer_id = 1)

```
SELECT id, day_of_week, start_time, end_time, available
FROM trainer_schedules WHERE trainer_id = 1;
```

Plan service

gympro_plan

All membership plans

```
USE gympro_plan;
SELECT id, plan_name, duration_type, price, duration_days, active
FROM membership_plans ORDER BY id;
```

Active plans only

```
SELECT id, plan_name, price, duration_type FROM membership_plans
WHERE active = 1;
```

All member subscriptions

```
SELECT id, member_id, member_email, plan_name,
start_date, end_date, status
FROM member_subscriptions ORDER BY id;
```

Active subscriptions only

```
SELECT id, member_email, plan_name, start_date, end_date
FROM member_subscriptions WHERE status = 'ACTIVE';
```

Subscriptions for a specific member

```
SELECT * FROM member_subscriptions WHERE member_id = 1;
```

Payment service

gympro_payment

All payments

```
USE gympro_payment;
SELECT id, member_email, amount, payment_method,
status, transaction_id, paid_at
FROM payments ORDER BY paid_at DESC;
```

Successful payments only

```
SELECT id, member_email, amount, transaction_id, paid_at
FROM payments WHERE status = 'SUCCESS';
```

Refunded payments

```
SELECT id, member_email, amount, transaction_id
FROM payments WHERE status = 'REFUNDED';
```

Find payment by transaction ID

```
SELECT * FROM payments
WHERE transaction_id = 'TXN-XXXXXXX';
```

Replace TXN-XXXXXXX with the actual transaction_id from the Postman response.

All payments by a specific member

```
SELECT id, amount, payment_method, status, transaction_id, paid_at
FROM payments WHERE member_id = 1;
```

Total revenue (successful only)

```
SELECT SUM(amount) AS total_revenue,
COUNT(*) AS total_transactions
FROM payments WHERE status = 'SUCCESS';
```

Quick cross-check — run after each Postman step

all databases

After POST /auth/register — verify user was created

```
USE gympro_auth;
SELECT id, name, email, role FROM users ORDER BY id DESC LIMIT 5;
```

After POST /members — verify member profile created

```
USE gympro_member;
SELECT * FROM members ORDER BY id DESC LIMIT 5;
```

After POST /trainers/schedule — verify slot added

```
USE gympro_trainer;
SELECT * FROM trainer_schedules ORDER BY id DESC LIMIT 5;
```

After POST /plans/subscribe — verify subscription

```
USE gympro_plan;
SELECT * FROM member_subscriptions ORDER BY id DESC LIMIT 5;
```

After POST /payments/pay — verify payment record

```
USE gympro_payment;
SELECT id, member_email, amount, status, transaction_id, paid_at
FROM payments ORDER BY paid_at DESC LIMIT 5;
```

After PUT /trainers/schedule/{id}/book — slot marked unavailable

```
USE gympro_trainer;
SELECT id, trainer_id, day_of_week, start_time, available
FROM trainer_schedules WHERE available = 0;
```